

18. DISEÑO ORIENTADO A OBJETOS: INTRODUCCIÓN

La Tierra es un lugar más bello para nuestros ojos que cualquiera que conozcamos. Pero esa belleza ha sido esculpida por el cambio: el cambio suave, casi imperceptible, y el cambio repentino y violento. En el Cosmos no hay lugar que esté a salvo del cambio.
Carl Sagan⁹⁴

En las metodologías estructuradas no se le prestó demasiada atención al diseño. En muchos casos se lo consideraba poco más que un epílogo del análisis. En otros, se hacía solamente un diseño detallado como etapa apenas previa a la programación. Incluso es abrumadora la diferencia cuantitativa entre libros publicados sobre análisis y sobre diseño de sistemas.

Aun en el marco de la orientación a objetos, hay metodologías que en los proyectos pequeños recomiendan juntar análisis y diseño y otras que proponen que el diseño y la programación los hagan las mismas personas y prácticamente al mismo tiempo. De todas maneras, en general esto no rebaja al diseño a una actividad menor: UP, por ejemplo, al aconsejar la no separación del análisis y el diseño en ciertos desarrollos, lo hace pensando en que el análisis – y no el diseño – puede ser minimizado. De la misma manera, XP, a la vez que sugiere que el diseño y la programación se hagan dentro del mismo equipo de programadores, enfatiza que el programador promedio debe ser un buen diseñador. Y es que en los métodos orientados a objetos el diseño es fundamental.

En este corto capítulo pretendo explicar a qué llamamos diseño en el marco de la orientación a objetos, y desterrar la noción errónea que lo desdeña, sea poniendo a la programación por sobre todo, o al análisis como disciplina más fundamental⁹⁵.

⁹⁴ Del cuatro episodio (Cielo e Infierno) de la serie Cosmos.

⁹⁵ Esto no quita que análisis y programación sean importantes, como he mostrado en capítulos anteriores.

Definición y objetivos

El término **diseño** lo hemos utilizado hasta el hartazgo en este libro. No es que esté pidiendo disculpas: al fin y al cabo, éste es un libro de diseño de software.

Pero ya es tiempo que definamos qué es el diseño.

Llamamos **diseño** – al menos los informáticos – a la actividad del desarrollo de software que se centra en la manera en la que resolvemos problemas.

Si pensamos en la resolución de un problema cualquiera, éste suele tener cuatro partes:

- La identificación de cuál es el problema que queremos resolver: el “qué”.
- El establecimiento de una modalidad de resolución, y de los elementos que necesitaremos para resolverlo: el “cómo”, incluyendo el “con qué”.
- La resolución del mismo.
- La validación y la verificación contra los requisitos.

En el desarrollo de software, estas tres actividades se suelen denominar:

- Relevamiento y análisis (*qué* vamos a realizar).
- Diseño (*cómo* y *con qué* lo vamos a realizar).
- Construcción o implementación, que incluye la programación (la realización del producto esperado).
- Pruebas (verificación y validación).

El diseño, en consecuencia, es la actividad más ingenieril del desarrollo de software, la que debe elegir herramientas de todo tipo – lenguajes, tecnologías, estructuras de datos, algoritmos, y muchas otras más – para resolver el problema, y debe determinar cómo utilizarlas en conjunto. Como se ve, es una actividad muy vasta. Como toda tarea ingenieril, es importante basarse en experiencias previas – propias o ajenas – y en prácticas de probada efectividad en la resolución de problemas similares.

El objetivo de todo diseño es construir un sistema en código que satisfaga las especificaciones de los clientes, que sea fácil de adaptar y modificar, y comunique la intención del diseñador y el programador a otros potenciales lectores. Siendo breve, debe ser funcional, flexible, adaptable y legible. Debe lograr flexibilidad manteniendo al mínimo la complejidad.

En esta parte del libro me voy a centrar en lo flexible y adaptable, que es el propósito central del diseño. De lo funcional me ocupo un poco en todos lados, aunque en último término es una cuestión centrada en el análisis. La legibilidad de los programas no la atendí de modo explícito, pero creo que todos los ejercicios resueltos del libro revelan esta inquietud.

Una vez definido el diseño como el “cómo” del desarrollo de software, queda pendiente definir los límites del diseño.

Muchos programadores dirán que ellos el diseño lo van haciendo a medida que programan, a partir de las especificaciones funcionales directamente. Y pueden perfectamente estar en lo cierto, sobre todo en proyectos medianos o pequeños. De hecho, XP propone algo en esta línea.

Otras personas – probablemente en organizaciones más formales – dirán que el diseño es una disciplina separada de la programación y que debe ser encarada por personas con roles diferentes.

También existirán personas que nos dirán que el diseño incluye cuestiones tan amplias y disímiles como la selección de las herramientas, lenguajes y plataformas de trabajo, la manera en que la aplicación deberá interactuar con otros sistemas, la decisión sobre nomenclatura en los programas, la definición de los estándares de diseño gráfico de la interfaz de usuario, los algoritmos a utilizar, etc. Todos ellos tienen razón: si bien en este libro nos centraremos sólo en algunos de estos aspectos, todo esto es diseño.

¿Es necesario el diseño?

Esta es una cuestión inadecuadamente polémica. La respuesta corta es sí. La larga...

Muchos programadores, sobre todo si han trabajado en proyectos pequeños, no ven al diseño como una actividad separada de la programación. Al fin y al cabo, si quiero resolver el problema puedo partir del relevamiento y construir el software sin más preámbulo. ¿Será así realmente? Creo que la clave está en el tamaño del proyecto. Hagamos una analogía con la construcción.

Si una tarde de domingo me levanto de la siesta con ganas de hacer una cucha para el perro, voy a la ferretería, compro unas tablas, un serrucho y clavos; regreso a casa y me pongo a construirla. Como nunca he hecho una cucha, probablemente me salga mal. No obstante, no me frustraré por ello: puedo desarmarla y volver a construirla. Y si realmente arruiné todo, puedo volver a ir a la ferretería, comprar de nuevo lo que necesite, e intentarlo nuevamente. En alguno de estos ensayos, llegaré a construir algo pasable, o abandonaré el proyecto sin mayores consecuencias que una pequeña frustración.

Si, en cambio, soy un ingeniero – lo soy, de hecho – que quiere construir una torre de oficinas de veinte pisos, no me tomaré las cosas tan a la ligera. Deberé analizar alternativas de ubicación, me reuniré con otros pares más algún arquitecto para definir el aspecto del edificio y su funcionalidad, elegiré el tipo de materiales a emplear, dibujaré planos o haré que los dibujen, planificaré el proyecto, conseguiré financiación, contrataré personal especializado en distintas tareas, y finalmente dirigiré la construcción del mismo. ¿Por qué me tomo todo este trabajo? Simplemente, porque las tareas a realizar son tan complejas y costosas que no me puedo dar el lujo de que salgan mal y deba hacer todo de nuevo.

¿Es válida esta analogía? Para algunos sí, y para otros no. Las opiniones están divididas, y hay muchos que sospechan que hay trampa en el razonamiento.

En realidad, trampa no hay. Lo que sí ocurre es que en el desarrollo de software, quienes hacen la tarea de construcción suelen ser técnicos y profesionales que a menudo están capacitados para las demás tareas también. Por eso, hay quienes han propuesto no hacer diseño previo a la construcción, sino dejar que el diseño evolucione gradualmente a partir del código. Otros, más escépticos y formales, critican esta visión, diciendo que esto aplica muy bien a pequeños desarrollos – como en la analogía con la cucha del perro – pero que es impracticable en proyectos medianos y grandes.

La verdad es que ambos pueden tener razón. El diseño evolutivo puede ser muy peligroso, ya que tiende a aumentar continuamente la entropía del código hasta hacerlo inmantenible e inmanejable. En un proyecto chico esto puede no ser un problema, pero las cosas se complican en sistemas reales. Sin embargo, la solución de realizar todo el diseño por adelantado, choca a menudo con algunas dificultades propias del desarrollo de software, la más grave de las cuales es el cambio de requisitos del sistema una vez que lo estamos construyendo. Y, en rigor de verdad, cuanto más grande es el proyecto en el que estamos embarcados, mayor es la probabilidad de que surjan cambios sobre la marcha. Y lo cierto es que varias décadas tratando de acotar el problema por el lado de impedir los cambios de requisitos, haciendo un concienzudo análisis previo, no han mostrado buenos resultados: los clientes quieren cambios, y son ellos los que pagan...

Pero hay algo que estamos dando por supuesto en nuestros razonamientos, que no tiene por qué ser así. Yo enuncié al diseño y la implementación como dos actividades distintas dentro del desarrollo de software, y mantengo mi postura. Pero que sean distintas no quiere decir, ni que deban realizarse separadamente, ni que deban realizarlas personas diferentes.

De hecho, si bien los primeros esbozos de métodos de desarrollo planteaban que antes de comenzar a escribir la primera línea de código había que tener concluida la actividad de diseño completa, hace ya mucho tiempo que hemos debido flexibilizar esta cuestión. Un poco tímidamente al principio, se empezó a pensar en desarrollos iterativos y evolutivos, en los cuales se realiza un poco de diseño, al que le sigue un poco de programación, seguido de otro poco de diseño, y más programación, y así sucesivamente. Con el tiempo, esto se fue formalizando, y surgieron métodos de desarrollo decididamente incrementales, así como otros que plantearon que el diseño y la programación se debían hacer a la vez por las mismas personas.

Lo importante es entender que un diseño se desarrolla a lo largo del tiempo: no es algo que haya que hacer de entrada y para siempre.

Otra cuestión sujeta a debate es cuál es el resultado del diseño. Hay quienes piensan el Diseño (con mayúsculas) como un conjunto de diagramas muy formales. En realidad, esto tampoco tiene por qué ser tan dogmático: se pueden usar diagramas para diseñar, pero también se puede diseñar en y mediante el propio código. Todo depende de lo que queramos hacer.

Todos estos enfoques son válidos, y ya volveré sobre ellos, pero lo importante es que ninguno oculta que existe una actividad de diseño, se haga a la-vez o no, con las mismas personas o no.

Qué diseñamos y cómo

El **diseño**, por lo tanto, y despejada ya la duda de su pertinencia, es la actividad del desarrollo de software que consiste en concebir una solución para un problema.

Entre las cuestiones que se definen en el diseño están:

- Subsistemas o componentes de software y sus interacciones.
- Despliegue en hardware de la solución.
- Necesidad de integración con otros sistemas.
- Interfaz de usuario.
- Estructuras de datos y algoritmos.
- Necesidad de persistencia y manera de realizarla: archivos, base de datos, etc.
- Definiciones tecnológicas: plataforma, lenguaje, base de datos, protocolos y dispositivos de conectividad, etc.

Distintas personas y equipos pueden llegar a distintos diseños para un mismo problema. Esto es así porque todo diseño depende de la experiencia del diseñador, de las prioridades del cliente y del equipo de desarrollo, de los requisitos no funcionales y de las posibilidades de la tecnología, entre otras cosas.

Quien hace el diseño suele ser una persona con el rol de diseñador. Muchas veces se trata de un programador con experiencia o un líder técnico del proyecto de desarrollo. Si todos los programadores deben diseñar, como plantean algunos métodos de desarrollo, hay que tener en cuenta que necesitamos personas de muy buen nivel técnico.

En algunos proyectos grandes aparece una especie de súper-diseñador, a menudo llamado “arquitecto”, que es responsable de los aspectos macro del diseño. El término no es muy feliz, como tantos de la informática, pero es bastante usual.⁹⁶

Sobre el tema de cómo debe realizarse el diseño, no hay una respuesta única tampoco.

Algunas recomendaciones van en la dirección de sugerir que se inserte la documentación de diseño directamente dentro del código.

Si deseamos separar un poco diseño y código, manteniendo una perspectiva ágil, es una buena idea almacenar las decisiones de diseño en una wiki del proyecto.

⁹⁶ Como ingeniero que soy, se imaginarán lo feliz que me hace que se denomine “arquitecto” a quien realiza la actividad más ingenieril del desarrollo de software, sobre todo teniendo en cuenta la especial relación entre ingenieros y arquitectos.

Las tarjetas CRC dan una posibilidad adicional, al permitir manipularlas, e incluso pueden ser digitalizadas.

Hay quienes han propuesto dibujar diagramas (UML o tarjetas CRC) en pizarrones y fotografiarlos con cámaras digitales, pudiendo luego almacenar estas fotos en el wiki del proyecto o una Intranet.

Otra opción más formal es trabajar con una herramienta integrada que almacena diagramas UML y permite generar código a partir de ellos, además de mantener la trazabilidad entre los distintos artefactos de desarrollo.

Diseño y complejidad

Debemos entender que el diseño es fruto de compromisos y prioridades. Una misma especificación funcional puede llevar a diseños distintos si las prioridades del cliente son diferentes, si las restricciones impuestas son otras, etc.

En última instancia, la razón de ser de un diseño es el manejo de la complejidad. De hecho, no hay aspecto más importante en el desarrollo de software que el control de la complejidad⁹⁷.

Una menor complejidad nos garantiza una mayor facilidad de comunicación, un menor costo de desarrollo y un sistema más flexible a los cambios.

Sencillez, claridad y elegancia son indicadores de buenos diseños; oscuridad, ingeniosidad y complejidad indican un diseño inadecuado y un pensamiento mal orientado. Toda complejidad tiene su costo.

Hay quienes han enfatizado esta necesidad diciendo que un diseño es perfecto cuando ya no se puede quitar nada del sistema sin comprometer los requisitos.

Diseño y cambio

Diseñamos, como ya dijimos anteriormente, para facilitar el cambio.

Ahora bien: no hay nada que moleste más a los desarrolladores que los cambios continuos. Al fin y al cabo, está muy instalada la idea de que los cambios de requisitos arruinan un diseño que costó mucho conseguir.

⁹⁷ Como corolario, el diseño debería ser esencial al desarrollo de software.

Sin que esto último deje de ser cierto, no perdamos de vista el propósito del diseño, para no caer en un círculo vicioso: diseñamos para permitir el cambio.

Hay algunas cuestiones de diseño que son más complejas de cambiar que otras. Algunas personas llaman a esto la arquitectura⁹⁸ de la aplicación. En estos casos, conviene ponerse de acuerdo en estas cuestiones lo más temprano posible en el proyecto de desarrollo, y estar atentos a los riesgos derivados de los posibles cambios. Pero no olvidemos que, aunque todo esté previsto, cambios va a haber: a lo sumo, podremos reducir su impacto, pero no impedirlos.

DISEÑO EN ESTE LIBRO

Nuestro foco en el diseño – en este libro – estará puesto más bien en las cuestiones que más afectan al desarrollo de software visto en su faz pura de software. Por lo tanto, no nos preocuparemos por las interfaces de usuario – a pesar de reconocer que es un tema importantísimo – ni por el diseño de base de datos – que daría para un libro entero – ni muchos de los otros aspectos antes enumerados.

Pero, aun visto desde nuestra perspectiva, el diseño puede ser interpretado de muchas maneras. O mejor diríamos, a muchos niveles. El cuadro que sigue muestra los diferentes niveles del diseño de software y las partes del libro en que los abordé:

Nivel de diseño	Descripción	Parte
Arquitectura macro o de contexto	Define la aplicación como un todo y su interacción con otras aplicaciones	1
Arquitectura o diseño de paquetes	Define los paquetes o subsistemas en que se va a dividir la aplicación y las comunicaciones y/o dependencias entre ellos	1 y 3
Diseño de clases	Define las clases que componen cada paquete y sus relaciones	2 y 3
Diseño "micro"	Define los aspectos internos de las clases, incluyendo los aspectos privados y algoritmos	2

Los niveles de clases y diseño micro fueron analizados en los capítulos de la parte previa, en los apartados denominados "cuestiones de diseño", que he colocado al final de cada capítulo, aunque el capítulo de patrones de esta tercera parte está centrado en diseño de clases. También los principios de diseño del próximo capítulo afectan al diseño de clases y al diseño de paquetes. Solamente hemos visto, en el capítulo de diseño de arquitectura, algo de los dos primeros niveles.

⁹⁸ Otra acepción de arquitectura, y van...

19. DISEÑO ORIENTADO A OBJETOS: PRINCIPIOS Y PROBLEMAS TÍPICOS

*Yo no estudio para escribir, ni menos para enseñar
(que fuera en mí desmedida soberbia), sino sólo por ver
si con estudiar ignoro menos
Sor Juana Inés de la Cruz⁹⁹*

La determinación de las clases y objetos de un sistema es la parte más difícil e importante del diseño. Pueden ayudarnos:

- Buenas prácticas de diseño y programación.
- Patrones de diseño.
- Una buena arquitectura.

A cada una de las primeras dedicaremos un capítulo. La tercera ya la analizamos en el capítulo 6.

Este capítulo se va a centrar en las buenas prácticas en general, aplicables tanto en diseño micro como a nivel más macro. La enorme revalorización del diseño desde la emergencia de la orientación a objetos ha provocado la proliferación de consejos, máximas, principios y patrones exitosos. De todo ello, incorporo aquí un extracto con los principios más importantes y qué cosas no hacer si queremos lograr una aplicación bien diseñada.

DISEÑO Y CLASES

En todo sistema, de las tareas de análisis y el diseño deben obtenerse como mínimo las clases, las interfaces – entendidas como firmas de métodos – y las relaciones entre clases. Las clases conceptuales o de dominio se descubren durante el análisis, por lo que en el diseño nos interesa refinarlas y hallar el resto. El diagrama de clases de UML es de mucha ayuda a estos efectos.

⁹⁹ De la “Respuesta a Sor Filotea de la Cruz”, por la religiosa y poetisa Sor Juana Inés de la Cruz (1648-1695), oriunda de Nueva España (México).

Encontrando clases de diseño

En la actividad de análisis hemos encontrado clases. Pero en la orientación a objetos una clase representa abstracciones de software relevantes para el sistema que se está construyendo.

Las fuentes más comunes de clases, luego del análisis, son:

- Las bibliotecas existentes: éstas son las más importantes, porque se reutilizan directamente.
- Intercambios de opiniones con otros diseñadores experimentados.
- Literatura sobre algoritmos y estructuras de datos.
- Literatura sobre patrones de diseño.

Las **clases de diseño** son aquellas que, si bien describen conceptos, como las clases de análisis, no pertenecen al dominio del problema sino al de la solución.

Por lo tanto, partimos de las clases de análisis (o modelo de dominio), pero a medida que trabajamos vamos a encontrar nuevas clases que obedezcan a cuestiones de diseño. A la vez, puede que nos demos cuenta de que algunas clases que tenían sentido en el análisis, no lo tienen en el diseño, y por lo tanto las eliminemos.

A menudo pensamos en clases como en entidades del dominio del problema, lo cual es bueno como comienzo y para tareas de análisis. Pero en el diseño, surgen clases por muchas otras necesidades.

Las razones que justifican la existencia de una clase son, entre otras (puse entre paréntesis la actividad involucrada en cada caso: A para análisis, D para diseño y C para construcción):

- Modelar una entidad del dominio (A)
- Modelar una entidad abstracta, de las que surgen de encontrar elementos en común entre varias clases, y nos llevan a construir una clase abstracta o interfaz cuando nos interese mantener la interfaz, o una clase delegada, cuando nos interesen los datos en común (D y A)
- Reducir complejidad, llevando a que haya una sola abstracción por clase (A y D)
- Aislar complejidad y detalles de implementación en una clase aparte (D y C)
- Aislar una porción de código que se espera que cambie respecto de otras que se suponen más estables (D)
- Agrupar operaciones relacionadas en una clase de servicios, como la clase *Math* de Java (D)
- Al refactorizar otra clase más compleja (D)
- Al separar también el código poco claro en clases especiales (D y C)

Diseño por contrato

Siempre que se diseñen clases, se debe separar conceptualmente, aunque físicamente sean la misma persona, al programador implementador del programador cliente.

Bertrand Meyer ha introducido la idea del diseño por contrato en su libro "Construcción de software orientado a objetos" [Meyer]. Él mismo se ocupó de dotar al lenguaje Eiffel de las herramientas necesarias para soportarlo, lo que permite una adecuada transición entre diseño y programación. Sin embargo, los demás lenguajes orientados a objetos, o bien no soportan las herramientas del diseño por contrato, o bien lo hacen parcialmente, lo cual limita un poco la aplicación del método. Como la idea es interesante, la vamos a analizar brevemente.

El método se basa en que existe un **contrato** entre implementador y cliente, y ambos esperan que la otra parte cumpla con él. Si esto es así, el comportamiento de una clase se puede especificar como si fuera un contrato. Adicionalmente, éste se materializa mediante *precondiciones* y *postcondiciones* de métodos, e *invariantes de clase*.

Usar el diseño por contrato es útil para chequear nuestro grado de conocimiento de la clase que se va a implementar. Como ocurre con la escritura de las pruebas antes del código, explicitar invariantes, precondiciones y postcondiciones, nos ayuda a prever buenos y malos usos de la clase. Además, si una clase está bien diseñada, su invariante se debe poder describir en una forma sencilla, lo cual sirve como testeo del diseño de clases. Pero la mayor ventaja de este modelo es que se centra en qué hacen las clases más que en cómo lo hacen.

A continuación muestro el uso del diseño por contrato para una clase que maneje conjuntos de elementos enteros:

Operación	Precondiciones	Postcondiciones
Constructor (max)	Ninguna	Queda construido un conjunto vacío que podrá tener hasta max elementos.
Union (B)	B y this están contruidos	En el resultado quedan todos los elementos de this más todos los elementos de B.
Interseccion (B)	B y this están contruidos	En el resultado quedan todos los elementos de this que son también elementos de B.
Diferencia (B)	B y this están contruidos	En el resultado quedan todos los elementos de this que no son elementos de B.
Pertenece (x)	this está construido, x es un número entre 0 y max	Devuelve verdadero si x pertenece al conjunto this.
Incluido (B)	B y this están contruidos	Devuelve verdadero si todos los elementos de B pertenecen también al conjunto this.

Vacio()	this está construido	Devuelve verdadero si el conjunto this está vacío.
Vaciar()	this está construido	El conjunto this queda vacío.
AgregarElemento (X)	this está construido	Si el elemento x no pertenecía al conjunto this, éste queda con un elemento más, de valor x.
SacarElemento (X)	this está construido	Si el elemento x pertenecía al conjunto this, éste queda con un elemento menos, el de valor x.
Cardinalidad()	this está construido	Devuelve la cantidad de elementos del conjunto.
Igual (B)	B y this están contruidos	Devuelve verdadero si todos los elementos que pertenecen a this pertenecen a B, y viceversa.

Cuando se implemente la clase *Conjunto*, las postcondiciones se deberían evaluar en los test unitarios, mientras que las precondiciones se deberían chequear al comienzo de los métodos, lanzando excepciones si no se cumplen.

Por lo tanto, en el marco del diseño por contrato, podemos construir clases escribiendo las pruebas unitarias de los métodos en base a las postcondiciones de los mismos, implementar excepciones para cuando no se cumplan las precondiciones, y escribir pruebas unitarias que chequeen esas excepciones.

Las precondiciones, dado que son una herramienta para el cliente, deben incluirse entre la documentación técnica. Las postcondiciones, en cambio, no necesariamente deben estar a disposición de los clientes, ya que algunas pueden depender de la implementación: habrá, por lo tanto, postcondiciones públicas y otras privadas. Los invariantes suelen ser casi siempre dependientes de la implementación.

Precondiciones y postcondiciones deben surgir de las especificaciones del método, por lo que no deben ser más ni menos estrictas que éstas.

El conjunto de todos los invariantes garantiza que el estado de un objeto es válido a través del tiempo, sin importar cuántos métodos se llamen ni en qué orden. Los objetivos principales de los invariantes son: servir como postcondiciones de constructores y verificar las restricciones de integridad, como los atributos vinculados.

Ya expliqué que el cumplimiento de los invariantes se debe dar entre sucesivas llamadas a métodos públicos, no necesariamente dentro de los métodos o entre métodos privados. Esto se debe a que son parte del contrato, y el contrato debe cumplirse ante un cliente, no internamente. Se dice que los momentos en que se cumple un invariante son *momentos estables*, y como mínimo deben ser: a la salida de un constructor, y antes y después de cada método no privado.

El diseño por contrato se hace con la idea de construir software correcto y tolerante a fallas, pues permite precisar bien los requisitos de clases y métodos. Sirve adicionalmente para aumentar la robustez y la confiabilidad del producto.

De allí que las precondiciones, postcondiciones e invariantes sirvan también como documentación del software, en las pruebas y en los controles de calidad.

PRINCIPIOS DE DISEÑO DE CLASES Y COLABORACIONES

Hasta ahora vimos en qué consiste el diseño, algunos enfoques generales y mucho de POO. Sin embargo, hay que entender mucho más que lo básico de POO para ser buenos diseñadores.

Según Martin [MartinUML], “un sistema está bien diseñado si es fácil de comprender, fácil de cambiar y fácil de reutilizar”.

En este ítem analizaremos algunos de los más importantes principios de diseño. Varios de ellos han sido elaborados por Robert Martin, aunque hay muchos sin un autor conocido. Cuando estudiemos patrones veremos la aplicación de estos principios en la práctica.

No repetir

El principio de diseño por excelencia es *no repetir*.

Esta idea de no repetir se refiere al código. Si dos trozos de código son iguales, hay que refactorizar para que quede uno solo, probablemente en un método.

También, al enunciar que no hay que repetir, nos referimos a código similar, que se puede colocar en un único método con los parámetros adecuados.

De la misma manera, habría que tratar de no repetir patrones de agrupaciones de clases, generalizando lo más posible.

¿Para que sirve no repetir? Un programador novato tal vez crea que lo que estoy recomendando es trabajar menos. Puede que en algunos casos sea así. Sin embargo, aun cuando ya el código repetido esté escrito, mi recomendación es refactorizar y unificar. Claramente, en este caso estoy proponiendo trabajar más.

Es que la recomendación de no repetir tiene que ver, no con la primera escritura, sino con el mantenimiento futuro. Normalmente, el código igual o similar se refiere a los mismos requisitos, y por lo tanto suele ser necesario cambiarlo junto. Y no hay un error más común, ante un cambio de requisitos, que cambiar en un solo lugar lo que debiera haberse cambiado en dos o más. Si el código en cuestión figura una sola vez, este peligro no existe.

Alta cohesión y bajo acoplamiento

En la programación modular existen dos características que hacen a la calidad de los módulos: la cohesión, que debe ser lo más alta posible, y el acoplamiento, que debe ser todo lo bajo que se pueda. Ya hablé de esto en el capítulo de delegación.

En POO, lograr alta cohesión y bajo acoplamiento *a nivel de métodos* es bastante sencillo. En efecto, los métodos son cohesivos casi por definición si seguimos las buenas prácticas, y el bajo acoplamiento suele ser fácil de lograr si se mantienen interfaces mínimas.

Sin embargo, habría que recordar que en POO los métodos no son el único módulo disponible. Las *clases son el módulo más importante en la orientación a objetos*, y ellas también deben cumplir los requisitos de acoplamiento y cohesión. Una clase tendrá alta cohesión cuando represente un tipo de objeto simple, o un agregado de tipos simples. Podemos garantizar una fuerte cohesión disminuyendo al mínimo las responsabilidades de una clase: si una clase tiene muchas responsabilidades probablemente haya que dividirla en dos o más. Del mismo modo, una clase tendrá bajo acoplamiento cuando tenga la menor interacción posible con otras clases. Esta dependencia significa que – si bien puede haber muchas clases que dependen de una – debería haber pocas dependencias hacia otras clases desde una sola. Para visualizar estas cuestiones, los diagramas de clases son herramientas fundamentales.

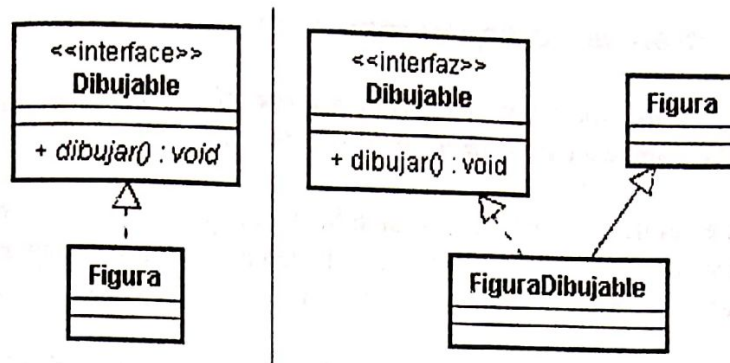
Otro módulo importante, aunque menos que la clase, es el *paquete*. Un paquete debe cumplir con estos mismos requisitos, sobre todo con el de acoplamiento débil, en el sentido de que debe tener vinculaciones mínimas con otros paquetes. En este caso, podemos ayudarnos con un diagrama de paquetes, que debido a que nos muestra dependencias entre conjuntos de clases, nos sirve para eliminar problemas de acoplamiento.

Única responsabilidad

Este principio fue enunciado por Tom De Marco [DeMarcoStructured] y Meilir Page-Jones [PageJones]. Según el mismo, cada clase debería tener una única responsabilidad. En general se lo alinea con el tema de cohesión, que implica que cada clase debe corresponder a una única abstracción.

Un corolario importante de este principio es el postulado de que cada clase tiene que tener una sola razón para cambiar.

Nótese el siguiente par de diagramas:



El de la derecha es más correcto, porque no agrega métodos de dibujo a *Figura*, y sin embargo hace que pueda haber variables del tipo *Figura* que puedan invocar al método *dibujar*. De hecho, las instancias de *FiguraDibujable* pueden usarse en todo el sistema como si fueran instancias de *Figura*.

Encapsulamiento de lo que varía

Este principio a veces es presentado como una consecuencia del anterior, y lo cierto es que están muy relacionados.

Según el mismo, hay que lograr que los cambios afecten a partes del sistema sin afectar al todo. La idea, entonces, es aislar los cambios. Con el mismo criterio, se suelen aislar las partes del sistema que estén desarrollados en otras tecnologías, paradigmas, las dependencias del hardware y las interfaces con otros sistemas.

Es un principio fundamental en una gran cantidad de patrones de diseño.

Principio abierto-cerrado

Este principio, que enunciara Bertrand Meyer [Meyer], expresa que las clases tienen que estar cerradas para modificación, pero abiertas para reutilización. Esto es, habría que tratar de no modificar clases existentes, ya probadas, estables y en producción. Pero esto no es óbice para que dichas clases puedan reutilizarse desde otras.

El uso más natural de este principio es la herencia, que permite agregar datos y comportamiento a clases existentes, aun cuando éstas estén cerradas. El problema con la herencia es que el agregado de comportamiento queda determinado en forma estática, en tiempo de compilación.

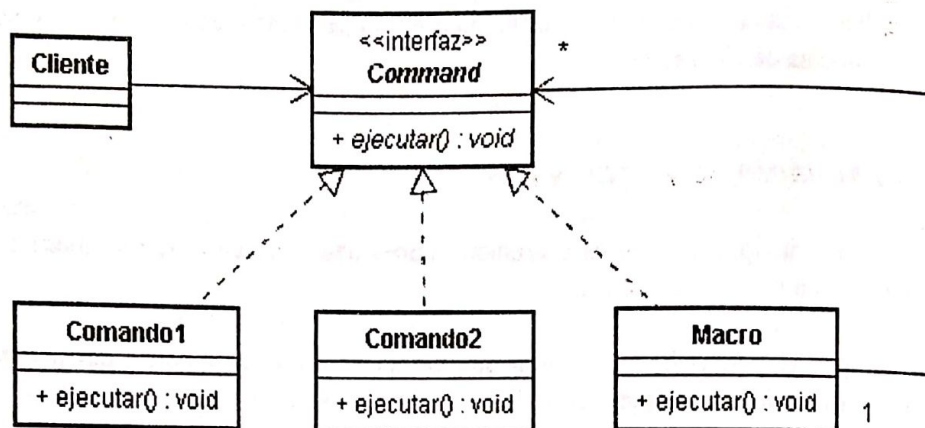
Mediante la delegación tenemos otra forma de reutilización, en este caso en forma potencialmente dinámica, ya que podemos decidir asociar un objeto a otro en tiempo de ejecución.

Minimizar la herencia de implementación

Este principio, que a veces se enuncia como “favorecer la delegación sobre la herencia”, llama bastante la atención de los más novatos en POO.

Ya dije en el ítem anterior que, si bien herencia y delegación sirven ambas para la reutilización, la herencia es estática, mientras que la delegación otorga mayor flexibilidad, al permitir diferir hasta el tiempo de ejecución el tipo de reutilización que se hará.

Veamos el diagrama del patrón Command que analizaremos en el próximo capítulo:



Acá vemos claramente que – gracias a la delegación – la elección del algoritmo (método *ejecutar*) se puede hacer en forma dinámica.

Por eso la herencia, a pesar de ser una herramienta importantísima de la POO, debe usarse con medida, y preferir siempre la delegación cuando sea una alternativa más o menos equivalente.

Un comentario aparte. Hay clases que poseen propiedades o métodos protegidos que necesitamos usar. En esos casos, tenemos tendencia a utilizar herencia. Sin embargo, en general, la mejor solución suele no ser tan lineal. Si no hay más razón para usar la herencia que la presencia del método o propiedad protegidos, lo mejor es crear una clase que herede de la clase objetivo, y nos provea su funcionalidad en forma pública o, mejor aún, con visibilidad de paquete. Luego, desde nuestra clase a construir, accederemos a la funcionalidad en cuestión por delegación.

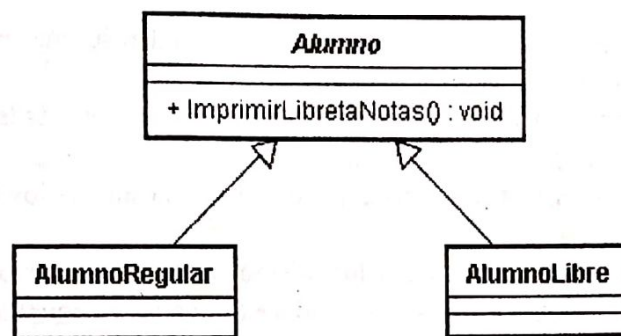
El próximo ítem, que habla del principio de sustitución, nos va a dar una nueva razón para evitar el uso indiscriminado de la herencia. Tanto es así que hay autores que han abogado por el no uso de herencia de clases concretas, lo cual – como todo extremismo – tampoco es la idea.

Sustitución de Liskov

Este principio, que lleva el nombre de quien lo enunció, Barbara Liskov [Liskov], plantea que los subtipos deben ser sustituibles en todas partes por sus tipos base. La aplicación más elemental de este principio es el chequeo de la relación “es un”, es decir, que la subclase sea un subconjunto de la clase. Pero hay situaciones más sutiles que hacen difícil discernir su buena aplicación.

La idea general es que las clases base no deben tener comportamientos que dependan de las clases derivadas, o incluso de su existencia.

Por ejemplo, supongamos que tenemos la siguiente jerarquía:



En este caso, ¿cómo deberíamos implementar *ImprimirLibretaNotas* en *AlumnoLibre*, suponiendo que un alumno libre no tiene las notas necesarias para colocar en la libreta?

Una primera solución – bastante pobre – es colocarle 0 en todas las notas a imprimir. Otra es arrojar una excepción cuando se llame al método, lo cual es bastante extraño, y al menos en Java nos obliga a declarar la excepción en la clase ancestro. Otra posibilidad es, en el método de la clase *Alumno* preguntar si el objeto referenciado por *this* es un *AlumnoLibre*, pero aquí la violación del principio de Liskov es flagrante: ahora la clase *Alumno* sabe de la existencia de la clase *AlumnoLibre*.

Para respetar el principio de sustitución, lo mejor sería dejar de considerar alumnos a los alumnos libres – y cambiar el nombre de alguna de las dos clases por algo que refleje esto – y de ese modo habremos solucionado el problema.

Para entender un poco mejor por qué surgen confusiones al estudiar el principio de sustitución, tal vez sea bueno recordar que la famosa relación “es un” se refiere al comportamiento y no a la estructura. Quizá sería buena idea testear, además de la relación “es un”, la relación “se comporta como”.

En el marco de este principio, hay autores que sostienen que lo que ocurre es que se confunde la relación tipo-subtipo con la relación clase-subclase. Afirman ellos que la relación tipo-subtipo es la que debe cumplir con la relación “es un”. La relación clase-subclase, en cambio, es una facilidad de la POO que busca representar la relación tipo-subtipo, y que no siempre lo logra bien. Por ejemplo, *int* es un subtipo de *double*, pero no suele estar implementado así por herencia. Dicen que a veces nos engañamos porque una definición nos invita a definir una herencia, como en: “una elipse es una circunferencia con un radio más”. Además, afirman estos autores, los compiladores no nos ayudan a encontrar estos errores conceptuales.

Creo que este planteo no contribuye demasiado, pateando el problema para otro lado. Lo importante es que tengamos en claro lo que estamos haciendo, y usar la herencia con cuidado.

Los corolarios del principio de sustitución son:

- Las precondiciones de un método no pueden ser más estrictas en una subclase de lo que son en su ancestro.
- Las postcondiciones de un método no pueden ser más laxas en una subclase de lo que son en su ancestro.
- Los invariantes de una clase deben ser al menos los mismos de la clase ancestro.
- Un método debe lanzar los mismos tipos de excepciones que en la clase ancestro, o a lo sumo excepciones derivadas de aquéllas.

Ya volveremos sobre este importante principio al estudiar el problema de la circunferencia y la elipse.

Inversión de dependencia

Lo que este principio pretende es no depender de clases concretas volátiles. Esto es, no conviene que una clase herede o tenga una asociación hacia una clase concreta que tiene alta probabilidad de cambio.

Como en general las clases abstractas y las interfaces son más estables, conviene utilizarlas para herencia o delegaciones, en vez de las clases concretas.

Este principio a veces es enunciado como: “ programe contra interfaces, no contra implementaciones”, aunque en este caso estamos poniendo el foco en el uso de un cliente.

Un ejemplo del uso de este principio es el que sigue:

```
Collection c = new ArrayList( );
Iterator i = c.iterator( );
```

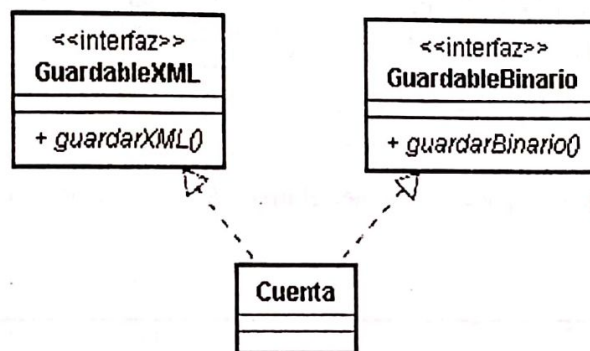
Notemos que en ambos casos, los tipos de las variables son interfaces: *Collection* e *Iterator*. Incluso el uso de la clase *ArrayList* podría evitarse con el uso de algún patrón de diseño de creación de objetos como los que veremos luego.

Segregación de la Interfaz

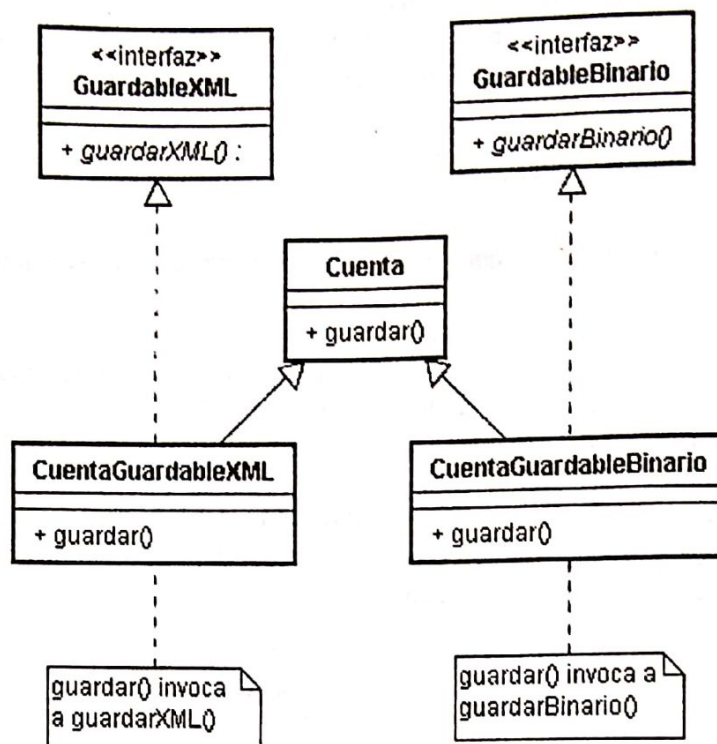
Este principio pretende que los clientes de una clase no dependan de métodos que no utilizan.

Así, si una clase tiene una referencia a, o hereda de, otra clase, de la cual sólo tiene sentido que utilice algunos de sus métodos, pero no todos, lo mejor sería separar la clase en cuestión en más de una, y colocar en una de ellas los métodos que utilice ese cliente.

Por ejemplo, supongamos que tenemos una clase denominada *Cuenta* con los métodos necesarios para guardar su estado en algún medio de almacenamiento persistente, en varios formatos. Un primer diagrama sería:



Normalmente, un mismo objeto no va a ser guardado en más de un formato. Una mejor solución, respetando el principio de segregación de la interfaz, sería la del siguiente diagrama:



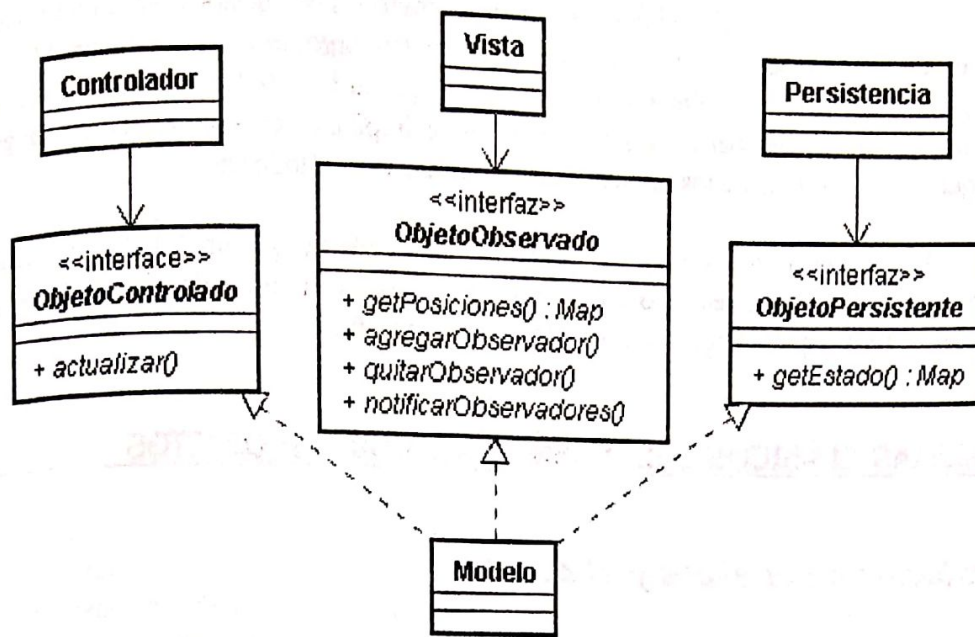
Pero, aun así, se puede mantener el nivel de abstracción usando solamente variables de tipo *Cuenta*:

```

Cuenta c = new CuentaGuardableXML();
c.guardar();           // llama a guardarXML()
  
```

Un buen ejemplo de este principio lo podemos ver en el diseño de una aplicación de simulación, a la que le dediqué un apéndice del libro.

El diagrama de clases simplificado es:



Nótese que la clase *Modelo* implementa tres interfaces, y que cada tipo de objeto la utiliza solamente a través de cierta interfaz. Esto hace, por ejemplo, que cuando un objeto *Controlador* utilice la clase, sólo pueda hacerlo llamando al método *actualizar*, mientras que un objeto *Persistencia* podrá invocar solamente al método *getEstado*.

Granularidad fina

La granularidad está definida por el tamaño de los objetos devueltos por los métodos y propiedades de una clase.

Por ejemplo, si en una clase *NominaEmpleados* tenemos un método *getEmpleado(dni)*, que nos devuelve todos los datos de un empleado, la granularidad es más gruesa que si tuviésemos métodos *getNombreEmpleado(dni)*, *getDomicilioEmpleado(dni)*, *getSueldoEmpleado(dni)*, y así sucesivamente.

El principio de granularidad fina exige que la granularidad de las interfaces de clases sean lo más finas que se pueda, sin violar el debido encapsulamiento. Esto es así porque una granularidad más fina permite acceder a datos más desagregados.

Hay una objeción que se plantea a menudo contra este principio, y se refiere a los datos que deben viajar entre distintas aplicaciones. En efecto, si deseo invocar métodos sobre una aplicación remota, me interesa que la aplicación proveedora me devuelva un objeto de grano grueso mediante la llamada a un solo método, en vez de obligarme a llamar a varios métodos de grano fino, pues el consumo de ancho de banda es mucho mayor en el segundo caso.

La objeción es real, y si deseamos que los métodos de nuestra aplicación se puedan invocar desde otras, deberíamos prestarle atención. Sin embargo, hay dos cuestiones a tener en cuenta. La primera, que normalmente no deseamos exportar toda la funcionalidad hacia otras aplicaciones, sino sólo una parte. La segunda, que nada impide tener algunos métodos de grano grueso que estén implementados mediante varias llamadas a métodos de grano fino.

Por lo tanto, lo que se puede hacer es tener toda la funcionalidad implementada con granularidad fina, proveyendo adicionalmente algunos métodos de grano grueso para invocación externa, que deleguen a los métodos de grano fino.

PROBLEMAS CLÁSICOS DEL DISEÑO ORIENTADO A OBJETOS

El problema de la elipse y el círculo

Este es uno de los casos emblemáticos de problemas del diseño orientado a objetos. Se lo conoce también como el caso del rectángulo y el cuadrado.

Tomemos el caso de las clases *Elipse* y *Circunferencia*. De nuestros cursos de Geometría sabemos que una circunferencia es un caso particular de elipse, por lo que siempre una circunferencia es sustituible por una elipse.

Sin embargo, una elipse tiene dos radios y puede formar un ángulo con el eje horizontal, propiedades que no tiene la circunferencia. Por eso, algunos programadores se sienten tentados de implementar a la clase *Elipse* como derivada de *Circunferencia*, agregándole un radio y un ángulo. ¿Es éste el proceder correcto? Claramente no, pues estaríamos violando el principio de sustitución de Liskov.

Pero no nos preocupemos demasiado, el error anterior es solamente un error de implementación, fácilmente solucionable. La clase *Elipse* que implementamos en el capítulo de colecciones admite una clase descendiente *Circunferencia*, que se podría definir así:

```
package carlosfontela.geometria;

public class Circunferencia extends Elipse {

    public Circunferencia (double radio, Punto centro) {
        super (radio, radio, centro, 0);
    }

    public double getRadio ( ) {
        return getRadioMayor();
    }
}
```

```
// se redefine solamente por dejar un código más claro:
public double getPerimetro ( ) {
    return ( Math.PI * 2 * getRadioMayor ( ) );
}

// se redefine solamente por dejar un código más claro:
public String getTipo ( ) {
    return "circulo";
}

}
```

Tal vez sea un poco artificial que la clase *Circunferencia* haya heredado las propiedades *getRadioMayor* y *getRadioMenor*; pero no es grave.

Podríamos preguntarnos qué ocurriría si agregamos a la clase *Elipse* métodos que nos devuelvan los puntos focales y la ecuación de la recta que pasa por los mismos. Tampoco esto es un problema serio, ya que podríamos redefinir esos métodos en *Circunferencia*.

```
public Punto getFoco (int i) {
    return Centro;
}

public double getPendienteRectaFocal ( ) {
    return 0;
}

public double getOrdenadaRectaFocal ( ) {
    return 0;
}
```

Incluso un cambio de escala en *Elipse* no debería traer problemas, ya que ni siquiera redefiniríamos el método en cuestión.

Podríamos incluso agregar un método *toCircunferencia* en la clase *Elipse*:

```
public Circunferencia toCircunferencia ( ) {
    if (radioMayor == radioMenor)
        return new Circunferencia (radioMayor, centro);
    else throw new UnsupportedOperationException ( );
}
```


Lo que sí es un tanto raro en este último caso es que necesitamos que la clase *Elipse* conozca a la clase *Circunferencia*, porque eso viola el principio de sustitución. En este mismo caso, podríamos incluso crear una instancia de *Circunferencia* cuando se intente crear una *Elipse* de radios iguales, aunque en este caso necesitaríamos trabajar encapsulando la creación de objetos con una especie de Factory Method¹⁰⁰.

Pero no nos pongamos contentos antes de tiempo. ¿Qué pasaría si en *Elipse* definimos un método *distorsionar* que permitiera cambiar la escala en uno solo de los ejes de la elipse? Ahora sí tenemos un problema, porque si bien una elipse distorsionada sigue siendo una elipse, una circunferencia distorsionada no es una circunferencia. Por lo tanto, la clase *Circunferencia* no puede cumplir sus invariantes y a la vez permitir el método *distorsionar*.

Una primera tentación será redefinir el método en cuestión en la clase *Circunferencia* para que arroje una excepción. Pero un corolario del principio de sustitución es que un método redefinido en una clase descendiente debe arrojar las mismas excepciones que en la clase ancestro, o a lo sumo excepciones derivadas de aquéllas. Además, elegir el lanzamiento de una excepción es renunciar al chequeo estático.

La solución más obvia es no permitir que el comportamiento de *distorsionar* exista en la clase *Elipse*. Claramente, una elipse matemática – si bien no es necesariamente inmutable – sí mantiene sus radios, aunque pueda trasladarse o girar.

Por lo tanto, podemos permitir generar una elipse distorsionada a partir de otra, incluso de una circunferencia. Pero siempre lo haremos creando una nueva elipse. El encabezado de dicho método, definido en la clase *Elipse* será:

```
public Elipse distorsionar (double factorRadioMayor, double factorRadioMenor) { ... }
```

Y su invocación podrá ser:

```
Elipse elipse = new Elipse ( 2, 4, new Punto (0, 0), 0 );
Elipse nueva = elipse.distorsionar (1.2, 0.8);
```

E incluso:

```
Circunferencia circunferencia = new Circunferencia ( 5, new Punto (0, 0) );
Elipse nueva = circunferencia.distorsionar (1.2, 0.8);
```

¹⁰⁰ Veremos el patrón Factory Method en el próximo capítulo.

Hay quienes opinan que hay que separar conceptos. Una elipse matemática no puede ser distorsionada, pero un objeto computacional, instancia de la clase *Ellipse*, sí podría. Personalmente, me parece un planteo absurdo. Si decimos que una de las ventajas de la POO es que permite representar directamente las entidades que estamos modelando, no deberíamos violar las leyes de esas entidades. A menos que nuestras circunferencias y elipses no sean entidades matemáticas, sino – por ejemplo – objetos dibujables con algunas propiedades en común con las elipses matemáticas.

También están los que opinan que podría haber una interfaz *Distorsionable*, que sea implementada por la clase *EllipseDistorsionable*. Ambas clases, *EllipseDistorsionable* y *Circunferencia*, derivarían de *Ellipse*, pero sólo *EllipseDistorsionable* tendría el método *distorsionar*. Se podría agregar un método que permitiera construir una elipse distorsionable a partir del estado de una circunferencia.

Una opción adicional es deshacerse de la clase *Circunferencia*, y dejar solamente una propiedad en la clase *Ellipse* que indique si una instancia particular de *Ellipse* es o no una circunferencia. Eso hice en el ejercicio inicial, en el que no preví una clase *Circunferencia*, pero sí devolvía esta condición en la propiedad *getTipo*.

Otro problema parecido – como dijimos más arriba – es el que surge entre rectángulos y cuadrados.

Pero estos problemas no ocurren sólo en la Geometría. Supongamos que tengo una clase que maneja préstamos hipotecarios y que tiene un método que permite la amortización adelantada: ¿cómo hago si no quiero heredar ese método porque en una nueva clase de préstamo que voy a derivar no deseo amortizaciones adelantadas?

El caso más general de este problema es el que [Meyer] denomina como *herencia de restricción*, en el cual la clase descendiente es similar a la clase ancestro, salvo por el agregado de una restricción en el invariante de clase.

Objetos que sólo ofrecen comportamiento: del principio al extremismo

Hay autores que opinan que los objetos debieran exponer sólo comportamiento y no comunicar su estado. Permitir acceder al estado de un objeto – mediante propiedades – es considerado por ellos como una violación del encapsulamiento. De hecho, dar a conocer el estado de un objeto implica dar a conocer el tipo de los datos que conforman el estado, lo que dificulta modificaciones posteriores de estos tipos.

Según esta línea de pensamiento, lo único permitido sería pedirles a los objetos que hagan tareas, no pedirles sus datos internos. Por ejemplo, una clase *Punto* debería tener métodos como *toString* o *imprimirAtributos*, pero no propiedades que permitan acceder a sus coordenadas *x* e *y*.

Este planteo está bien alineado con la teoría de orientación a objetos. De hecho, utilizar a los objetos enviándoles mensajes y esperando sus respuestas, es lo más ortodoxo que se puede esperar, y además respeta el encapsulamiento al máximo, permitiendo cambios en la manera de implementación del estado de los objetos.

Pero como todo planteo demasiado ortodoxo, puede ser extremista. No es tan raro que se necesite conocer el estado de un objeto.

Por ejemplo, veamos el método *imprimirAtributos*, que supuestamente es una solución orientada a objetos. Sin embargo, la implementación del método en cuestión va a estar muy ligada al tipo de representación y el soporte en el cual se debe imprimir el estado. Nuestro interlocutor ortodoxo afirmará sin ruborizarse que *toString* no tendría ese problema. Pero no es tan así: no siempre se necesita la representación en texto plano, ¿qué ocurre si deseamos también el objeto en XML? Poner dos métodos, *toString* y *toXML*, empieza a ser más cuestionable, pues viola el principio de segregación de la interfaz. La solución, me dirán entonces, es separar dos clases *PuntoRepresentableXML* y *PuntoRepresentableString*. Pero, ¿cómo se supone que hace cada una de estas clases para representar un punto sin poder acceder al estado contenido en otra clase?

Además, hemos visto que las propiedades no se supone que expongan el estado tal cual está implementado en forma de atributos, o al menos no necesariamente.

La habilidad del diseñador consistirá en exponer – como propiedades – el estado en una manera que sea luego susceptible de modificar si cambia la implementación, sin afectar a los clientes. Lo que está mal es que cada atributo de una clase tenga una propiedad de lectura y escritura. Las propiedades son una especie de atributo conceptual, independiente de la implementación.

¿Hay que evitar la herencia de Implementación?

Hay otro planteo, un poco más elaborado que el anterior, que también está provocando algo de inquietud entre los diseñadores menos experimentados. Si lo deseable es programar contra interfaces, y preferir la delegación a la herencia, ¿está mal la herencia de implementación?

Hay autores que opinan que sí. Un artículo en la Web [Holub], titulado “Why extends is evil” ha logrado una popularidad inesperadamente grande. En la práctica, lo que plantea esta línea de pensamiento es que todas las clases concretas deberían ser declaradas final.

Creo que se trata de otra idea extremista basada en un buen principio de diseño.

No me queda duda de que la herencia entre clases concretas acopla la clase derivada a la ancestro. Al fin y al cabo, es una forma de composición. También soy consciente de que cada vez que modifico una clase base, debo verificar las derivadas porque pueden funcionar mal. Incluso debo comprobar el código que use la clase básica y las derivadas, con la

complicación adicional de que a veces ni siquiera dispongo de las clases descendientes. Todo esto aporta pruebas de la fragilidad de las clases básicas concretas. Hasta estoy dispuesto a admitir que el patrón Template Method es peor que el Estrategia por estas razones, y por eso es menos dinámico¹⁰¹.

Asimismo, es cierto que no tiene sentido usar herencia de implementación si vamos a redefinir todos los métodos del ancestro. Si un ancestro existe para redefinir todos sus métodos, debería ser una interfaz.

Finalmente, también debo admitir que la POO no necesita de la herencia de implementación si se propone lograr encapsulamiento, uso de comportamiento de objetos y polimorfismo.

Pero aun sabiendo todo esto, creo que eliminar la herencia de implementación nos obligaría a reescribir mucho código, y la repetición de código provoca muchos más problemas potenciales que los problemas que estamos enunciando aquí.

El mismo patrón Template Method ha sido la base de muchos frameworks que funcionan muy bien y nos ahorran mucho trabajo.

Como siempre, hay que usar los principios en su justo término.

Toda vez que podamos, debemos depender de interfaces o clases abstractas. Pero cuando no se pueda, la herencia de implementación es una opción válida.

Es importante usar bien las clases base, siguiendo las intenciones del que las provee. Claramente, no conviene depender de comportamiento de las clases que no estén explícitas en el contrato. A la vez, el que provee clases para ser heredadas tiene que tener el cuidado de mantener el contrato explícito y los supuestos implícitos a través de las versiones. De todas maneras, esto último es fundamental también cuando se usan interfaces, porque las interfaces son por definición entidades sin implementación, así que el respeto del contrato es central.

Y es obvio que no es una buena idea usar atributos protegidos, porque violan el encapsulamiento.

PRINCIPIOS DE DISEÑO MACRO

Si bien escribí un capítulo completo sobre diseño arquitectónico en la primera parte del libro, hay algunos principios adicionales al de separación de incumbencias que vimos allí, y que se refieren a paquetes de clases. Además, como son base también para la separación de incumbencias, aquí es donde se sientan las bases para aquel capítulo, presentado con tanta anticipación.

¹⁰¹ En el próximo capítulo veremos estos patrones.

Diseño de paquetes: recomendaciones

Cuando hablamos de diseño, más allá de las clases y sus relaciones, el concepto que aflora naturalmente es el de paquete. Los paquetes permiten un nivel de abstracción más alto que las clases, de modo tal que es usual que, cuando deseamos tener una visión más global de un sistema, pensemos en los paquetes que lo componen.

De allí que sea fundamental diseñar bien los paquetes y sus interacciones, para que ese nivel alto de abstracción sea más comprensible, y por lo tanto, más mantenible y escalable.

El principio del bajo acoplamiento, ya analizado, adquiere una fundamental relevancia en el diseño de paquetes. Decimos que hay dependencia entre paquetes cuando hay clases de un paquete que dependen de clases de otro paquete, sea por herencia, asociación o simple dependencia débil.

Algo de dependencia entre paquetes es obviamente inevitable, ya que las dependencias entre clases en muchos casos cruzan los límites de los paquetes. Hay quienes pretenden que las relaciones de herencia no deben cruzar estos límites, pero es una idea ingenua, y no ha demostrado ser útil.

Lo que sí hay que lograr es que los cruces de dependencias entre paquetes sean la menor cantidad posible. El diagrama de paquetes de UML es una herramienta invaluable para analizar dependencias.

Otra recomendación habitual de diseño es construir las agrupaciones de clases de modo tal que sirvan para reutilizarse. Al fin y al cabo, los clientes reutilizan mucho más los grupos de clases que las clases aisladas. Por lo tanto, las clases que se prevé que se van a usar conjuntamente deberían colocarse en el mismo paquete. Asimismo, siguiendo en parte el principio de segregación de interfaz, conviene separar los paquetes por los clientes que los usan.

De la misma manera, si aplicamos algo similar al principio de única responsabilidad, las clases que cambian poco deberían ubicarse en paquetes separados de aquellas que cambian mucho.

También es deseable evitar las referencias cíclicas entre paquetes. Para esto contamos con algunas herramientas que nos ayudan a encontrar problemas, aunque el diagrama de paquetes suele ser invaluable.

Siguiendo el principio de inversión de dependencia, los paquetes de los cuales dependen muchos otros deben contener la mayor cantidad posible de clases abstractas e interfaces. Esto tiene como consecuencia que un paquete no dependa de otro menos estable. Otra forma de ver lo mismo es que cada paquete debe tener menor probabilidad de cambio que el paquete que depende de él.

Diseño de componentes

Cuando se habla de diseño arquitectónico o macro aparece la noción de *componente*. Sin embargo, en este libro he sido bastante cuidadoso, usando poco este término, o bien lo he usado sin preocuparme demasiado del significado.

Un **componente** de software es algo que se puede distribuir a nuestros clientes. En principio, es la unidad mínima que se puede desplegar en el hardware destino. Es – por lo tanto – un término ambiguo y equívoco, muy dependiente de la plataforma.

Sin embargo, lo cierto es que un componente es algo con una entidad más física que una clase o paquete. Esto es obvio desde el momento en que lo definimos como un entregable.

Tal vez lo bueno sea definir ejemplos de componentes. En la plataforma Java, podemos distribuir clases compiladas, con lo cual el componente coincide con la clase, aunque algunas personas asocian los componentes a las agrupaciones de clases que se distribuyen como archivos comprimidos tipo *JAR*, *WAR* y afines.

De todos modos, definamos como definamos a los componentes, los principios de diseño que se le pueden aplicar son los mismos de los paquetes, aunque tal vez debamos ser más estrictos en cuanto a dependencias, por la necesidad de compilar o recompilar ciertas partes.

CONCLUSIONES

Una buena forma de recapitular sobre los principios de buen diseño es analizar qué cuestiones hacen malo a un diseño.

Hay varias razones por las cuales un diseño puede ser malo. Lo que sigue son algunas señales, según [MartinUML]:

- Rigidez: dificultad de cambiar, porque cada vez que se modifica algo, esto provoca una secuencia interminable de cambios.
- Fragilidad: cada cambio provoca problemas en partes del código que no tienen que ver con el mismo.
- Se hace muy difícil descomponer el sistema en partes reutilizables.
- El ciclo editar-compilar-probar lleva mucho tiempo.
- Complejidad innecesaria: estructuras de código que no se necesitan, aunque puedan servir en el futuro.
- Repetición innecesaria: no se reutiliza, sino que se ha copiado partes de código en varios lugares.
- Opacidad: el programador se expresa – mediante el código – de formas ininteligibles.

Hay otros problemas típicos de diseño, que Martin Fowler¹⁰² [FowlerRefactoring] llama “malos olores”, que los buenos diseñadores enseguida detectan con su intuición. A continuación enumero algunos de los olores más usuales:

- Ciclos muy anidados, que se deberían convertir en métodos.
- Código duplicado, que causa modificaciones paralelas.
- Métodos complejos o muy largos.
- Clases con varias responsabilidades, o muy grandes.
- Abundancia de sentencias switch o if anidadas.
- Largas secuencias de llamadas sucesivas a métodos, como en:
`a.m1().m2().m3();`
- Demasiados chequeos de referencias nulas.
- Clases sin comportamiento, que sólo tienen atributos y las propiedades que permiten acceder a ellos.
- Atributos no encapsulados.
- Métodos que realizan más llamadas a mensajes sobre otras clases que sobre la suya propia.
- Uso de tipos primitivos o básicos para conceptos diversos: *double* en vez de *Dinero*, *Vector3D* en vez de *Velocidad*, etc.
- Comentarios que explican código difícil de leer.

Ahora que conocemos los principales requisitos del diseño orientado a objetos, el próximo capítulo abordará el tema de patrones de diseño, formalizando algunos que ya conocemos y presentando otros pero, sobre todo, analizando cómo estos patrones cumplen los principios recientemente vistos.

¹⁰² En su libro Refactoring, Fowler dice que la idea la tomó de Kent Beck, pero yo la lei de él.